Evaluation of Agentic – RAG Architectures

Ioannis Giannakos

Διπλωματική Εργασία

Επιβλέπουσα: Evaggelia Pitoura

Ιωάννινα,

Φλεβάρης 2026

ΤΜΗΜΑ ΜΗΧ. Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY OF IOANNINA

Περίληψη

Η παρούσα διπλωματική εργασία παρουσιάζει τον σχεδιασμό και την υλοποίηση ενός προηγμένου συστήματος Retrieval-Augmented Generation (RAG), βελτιστοποιημένου για τον ιατρικό τομέα, με στόχο την ενίσχυση της ακρίβειας και των συλλογιστικών ικανοτήτων των Μεγάλων Γλωσσικών Μοντέλων (LLMs) σε πολύπλοκα κλινικά ερωτήματα.

Σε αντίθεση με τις συμβατικές προσεγγίσεις RAG, το προτεινόμενο σύστημα υιοθετεί μια Agentic αρχιτεκτονική. Μέσω της ενσωμάτωσης αυτόνομων Agents σε όλα τα στάδια της διοχέτευσης δεδομένων (pipeline), η διαδικασία ανάκτησης μετατρέπεται από μια γραμμική αναζήτηση σε μια δυναμική, συλλογιστική διεργασία.

Ειδικότερα, εισάγεται μια εξειδικευμένη διαδικασία επεξεργασίας δεδομένων που αξιοποιεί έναν "Agentic Chunker" για τη διάσπαση ιατρικών κειμένων σε νοηματικά πλήρεις ενότητες (chunks), οι οποίες ευρετηριάζονται σε βάση FAISS, με στόχο την πιο άμεση αναζήτησή τους. Η αρχιτεκτονική ανάκτησης ακολουθεί μια ροή πολλών σταδίων. Αρχικά, ένας Agent σχεδιασμού (Planner) αποδομεί τα ερωτήματα, στη συνέχεια η μονάδα ανάκτησης συγκεντρώνει υποψήφια έγγραφα και τέλος, πραγματοποιείται φιλτράρισμα και ανακατάταξη (re-ranking) των ανακτηθέντων εγγράφων.

Για την επικύρωση του συστήματος, αναπτύχθηκε ένα πλαίσιο αυτοματοποιημένης αξιολόγησης που ενσωματώνει έναν Agent κριτή (Judge Agent) για την ποσοτική εκτίμηση της συλλογιστικής ποιότητας και των παραισθήσεων (hallucinations). Η συγκριτική ανάλυση αποκαλύπτει ότι, ενώ η συνολική ακρίβεια παραμένει συγκρίσιμη με το βασικό μοντέλο (το οποίο λειτουργεί χωρίς εξωτερική πληροφορία), η προτεινόμενη αρχιτεκτονική μειώνει σημαντικά τις παραισθήσεις σε εξειδικευμένα σενάρια και παρέχει εγκυρότερη τεκμηρίωση.

Λέξεις Κλειδιά: Ανάκτηση-Ενισχυμένη Παραγωγή (RAG), Μεγάλα Γλωσσικά Μοντέλα (LLMs), Τεχνητή Νοημοσύνη, Αυτοματοποιημένη Αξιολόγηση, Πράκτορες Τεχνητής Νοημοσύνης

Abstract

This thesis presents the design and implementation of an advanced Retrieval-Augmented Generation (RAG) system, optimized for the medical domain, aiming to enhance the accuracy and reasoning capabilities of Large Language Models (LLMs) in complex clinical queries.

In contrast to conventional RAG approaches, the proposed system adopts an Agentic architecture. Through the integration of autonomous Agents at all stages of the pipeline, the retrieval process is transformed from a linear search into a dynamic, reasoning process.

Specifically, a specialized data processing pipeline is introduced that utilizes an "Agentic Chunker" to split medical texts into semantically complete chunks, which are indexed in a FAISS database for more efficient retrieval. The retrieval architecture follows a multi-stage workflow. Initially, a planning Agent (Planner) decomposes the queries; subsequently, the retrieval unit gathers candidate documents, and finally, the retrieved documents are filtered and re-ranked.

For system validation, an automated evaluation framework was developed that incorporates a Judge Agent for the quantitative assessment of reasoning quality and hallucinations. Comparative analysis reveals that, while overall accuracy remains comparable to the baseline model (which operates without external information), the proposed architecture significantly reduces hallucinations in specialized scenarios and provides more valid justification.

Keywords: Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), Agentic Workflow, Automated Evaluation, Agentic Chunker, Agentic Planning, Agentic Retrieval, Agent as Judge

Table of Contents

FIGURES

TABLES

Κεφάλαιο 1. Introduction

1.1 Introduction

In recent years, we have witnessed a revolution in Natural Language Processing (NLP) driven by the rapid advancement of Large Language Models (LLMs). We observe that these models can generate coherent text and perform complex reasoning tasks with remarkable fluency.

However, we recognize that these parametric models suffer from inherent limitations, most notably their inability to access real-time information and their tendency to hallucinate facts when operating outside their training distribution.

To address these issues, we adopt the Retrieval-Augmented Generation (RAG) architecture, which grounds LLMs in external, verifiable knowledge bases. Nevertheless, we find that the standard "Naive RAG" approach, characterized by linear, fixed chains of execution, often fails in complex, domain-specific scenarios. We identify that standard pipelines lack self-correction mechanisms and rely on arbitrary text segmentation (chunking) that destroys semantic coherence.

In this thesis, we present the design and implementation of a Fully Agentic RAG Pipeline. We shift from rigid execution chains to autonomous, decision-making agents.

In the context of Large Language Models (LLMs), an **agent** is defined as an autonomous system that utilizes an LLM as its central reasoning engine to plan and execute multi-step tasks. Unlike passive models that simply generate text based on input prompts, an agent possesses the capability to perceive its environment, decompose complex problems into manageable sub-tasks, and interact with external tools, such as databases or search APIs, to achieve specific objectives dynamically.

We introduce innovations at multiple stages of the pipeline: from "Agentic Chunking" during data ingestion to an "Agentic Planner" and a "Judge Agent" during execution and evaluation. Moreover, we utilize a challenging medical corpus (StatPearls) as a rigorous testbed to validate the system's capacity for handling high-complexity, unstructured data.

1.2 Aim and Objectives

Our primary aim is to architect, implement and evaluate a robust Agentic Workflow for Retrieval-Augmented Generation (RAG) within the medical domain. Unlike traditional, linear RAG systems that rely on static retrieval and often fail to capture complex dependencies, our goal is to demonstrate how autonomous agents can dynamically reason over retrieved data. By bridging the gap between raw information retrieval and clinical reasoning, we aim to minimize hallucinations and enhance the factual accuracy of Large Language Models (LLMs) when addressing complex medical queries.

More specifically, we have defined the following core objectives:

1. Engineering an Agentic Data Ingestion Pipeline: We aim to fundamentally improve how medical knowledge is processed before indexing. Traditional methods (like fixed-size chunking) often fragment context, leading to information loss.

- **Objective**: To develop a specialized "Agentic Chunker" powered by Llama-3.
- **Methodology**: This agent will autonomously parse unstructured medical texts and decompose them into atomic, semantically complete propositions. This ensures that each indexed chunk carries a self-contained meaning, thereby maximizing retrieval precision.

2. Designing a Multi-Agent Retrieval Architecture: We seek to transform the retrieval process from a simple keyword search into a structured planning task.

- **Objective**: To implement a modular Multi-Agent System that separates query understanding from evidence gathering.
- **Methodology**: We will employ a Planning Agent responsible for analyzing user queries and decomposing them into logical sub-tasks. These sub-tasks will be executed by a Retrieval Module, which dynamically aggregates evidence from the vector store, allowing the system to synthesize answers based on multi-faceted reasoning rather than single-document lookups.

3. Establishing an Automated Evaluation Framework: We aim to move beyond subjective human evaluation by creating a scalable, quantitative benchmarking process.

- **Objective**: To create a sophisticated "Judge Agent" capable of auditing the system's performance.
- **Methodology**: This agent will be designed to assess reasoning quality and factuality by comparing the system's outputs against the ground truth and a non-RAG baseline. The framework will specifically focus on detecting hallucinations and measuring the system's adherence to the retrieved context, providing a clear metric of the architectural improvements.

1.3 Theoretical Background

1.3.1 Limitations of Naïve RAG

We begin by analyzing the standard "Naïve RAG" implementation to identify the critical bottlenecks that necessitate an Agentic approach. Naïve RAG follows a strict, linear pipeline: *Retrieve-Read-Generate*. This monolithic process treats all user queries uniformly, regardless of complexity.

Our analysis identifies three primary failure modes in this architecture:

1. **Retrieval Sensitivity & "Context Poisoning":** Standard retrieval relies on vector similarity (Cosine Similarity). However, irrelevant documents that share superficial keywords with the query can be retrieved, "poisoning" the context window. The LLM, forced to use this irrelevant context, often hallucinates to bridge the gap between the query and the retrieved noise.
2. **Lack of Multi-Step Reasoning:** Complex clinical queries (e.g., *"Compare the side effects of Drug A and Drug B for diabetic patients"*) require multi-hop reasoning. A Naïve RAG system performs a single retrieval step, often missing the comparative aspect or retrieving information for only one part of the question, leading to incomplete answers.
3. **Absence of Self-Correction:** The linear nature of Naïve RAG is a "one-shot" process. If the retrieval step fails, the generation step has no mechanism to signal a failure, adjust the query, and re-attempt the search.

1.3.2 The Shift to Agentic Workflows

To overcome these limitations, we embrace the paradigm shift from static chains to Agentic Workflows. In this context, an "Agent" is not merely a script, but a Large Language Model (LLM) configured to function as a cognitive reasoning engine.

Core Architecture of LLM Agents: Under the hood, these Agents are powered by the Transformer architecture. This deep learning framework revolutionized Natural Language Processing by dispensing with recurrence entirely. The key difference from traditional models lies in how they process information: while older networks (like RNNs) read a sentence sequentially, from left to right, Transformers utilize a mechanism called 'Self-Attention' to view the whole sentence simultaneously. This allows the model to weigh the importance of each word and understand connections between distant parts of a sentence, leading to superior comprehension and reasoning capabilities.

While the base function of an LLM is probabilistic token prediction (predicting the next word based on context), Agentic behavior emerges through advanced prompting strategies such as **Chain-of-Thought (CoT)** and **ReAct (Reason + Act)**:

- **Chain-of-Thought:** The model is guided to generate intermediate reasoning steps before the final answer, effectively miming human deduction.

- **The ReAct Pattern:** The Agent enters a loop of *Thought -> Action -> Observation*.
    1. **Thought:** The Neural Network analyzes the user's request and decides it needs external information.
    2. **Action:** The Agent generates a specific syntax to call a tool (e.g., Search("Symptoms of X")).
    3. **Observation:** The tool executes and returns raw data to the Agent's context window.
    4. **Reflection:** The Agent processes this new data and decides if it is sufficient or if a new action is required.

This cyclic process transforms the LLM from a passive text generator into an active problem-solver capable of planning and error correction.

1.3.3 Semantic Data Processing : Beyond Mechanical Chunking

A critical, often overlooked component of RAG efficacy is the quality of the indexed data. We propose that the limitation of many systems lies in Mechanical Chunking.

The Problem with Fixed-Size Chunking: Traditional methods split text based on character counts (e.g., every 500 characters). This is arbitrary and semantically destructive. For example, a pronoun ("He") might end up in one chunk, while the entity it refers to ("Dr. Smith") remains in the previous chunk. When the vector embedding is created for the second chunk, the meaning of "He" is lost, leading to low-quality vector representations.

Proposed Solution: Semantic/Agentic Chunking

Our thesis introduces a **Proposition-Based Indexing** strategy. We employ a smaller, specialized LLM during the ingestion phase to process the raw text. Instead of cutting the text blindly, the Agent:

1. **Analyzes** the text to identify distinct logical statements.
2. **Rewrites** these statements into standalone, atomic propositions.
3. **Resolves Coreferences:** It replaces pronouns (e.g., "it", "they") with their specific nouns, ensuring each chunk is self-contained.

Impact on Vector Space: By ensuring that each chunk represents a complete idea, we significantly improve the Semantic Density of the resulting vector embeddings. When a user queries the system, the vector similarity search is performed against clear, complete facts rather than fragmented text segments, drastically improving retrieval precision.

1.4 Thesis Structure

We organize the remainder of this thesis as follows:

- In **Chapter 2**, we detail the system architecture, focusing on the prompt engineering and logic behind the Chunker, Planner and Retrieval components.
- In **Chapter 3**, we present our experimental setup and evaluation methodology, where we utilize the Judge Agent to benchmark our system against a baseline LLM.
- In **Chapter 4,** we summarize our findings and discuss the scalability of agentic RAG systems to other domains. Finally, we propose directions for future research, focusing on addressing identified limitations and enhancing the robustness of the evaluation framework.

Κεφάλαιο 2. System Architecture

2.1 System Overview

This chapter delineates the architectural design and implementation of the proposed Retrieval-Augmented Generation (RAG) system. The system is structured as a modular, multi-stage pipeline designed to address the limitations of static retrieval methods.

In contrast to conventional RAG frameworks, which typically restrict the Large Language Model (LLM) to a terminal role, serving primarily as a text generator, the proposed architecture integrates agentic reasoning throughout the entire workflow. This approach shifts the reliance from passive execution to active decision-making, influencing both how data is prepared and how the final response is synthesized

The operational workflow is organized into two distinct phases:

1. **The Offline Phase (Data Ingestion):** This phase concerns the preprocessing and transformation of raw medical corpora into semantic vector embeddings, ensuring data is structured for optimal retrieval prior to user interaction.

2. **The Online Phase (Inference):** This phase manages the real-time processing of user queries, orchestrating a dynamic sequence of planning, information retrieval, and context-aware generation.

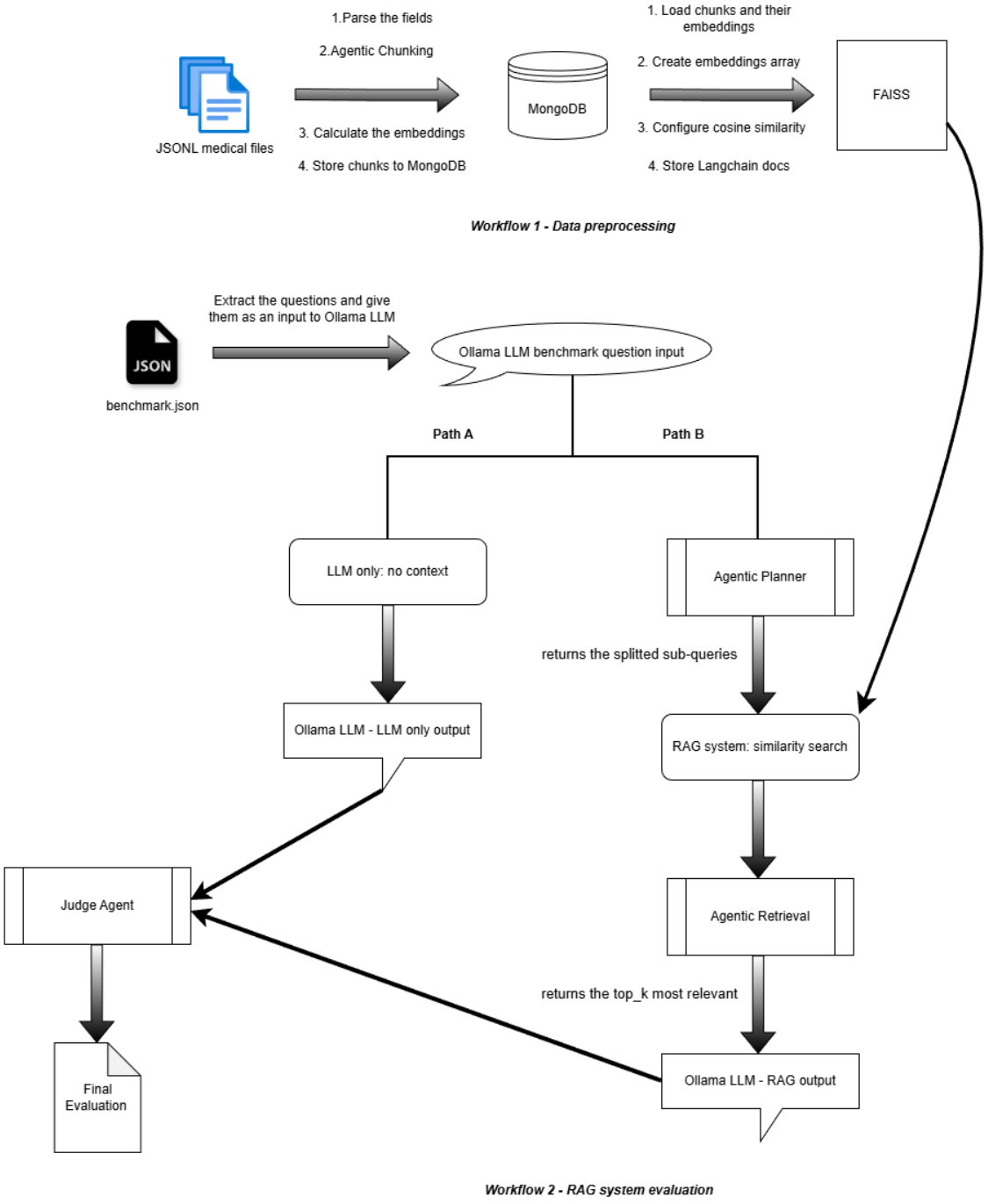

Figure 1. System Workflow

Based on the provided diagram, the architecture of the system is divided into two primary, interconnected workflows: Data Preprocessing and RAG System Evaluation.

Workflow 1: Data Preprocessing

This workflow is responsible for ingesting raw medical data and preparing it for efficient retrieval.

- **Data Ingestion and Chunking:** The process begins with "JSONL medical files" as the source data. The fields are parsed, and the data undergoes "Agentic Chunking". Following chunking, embeddings are calculated, and these chunks are stored in a "MongoDB" database.
- **Vector Indexing:** The system loads the chunks and their corresponding embeddings from MongoDB to create an embeddings array. It configures "cosine similarity" as the distance metric and stores the processed Langchain documents into "FAISS," which serves as the system's vector database for similarity search.

Workflow 2: RAG System Evaluation

This workflow evaluates the performance of the RAG system against a baseline by processing benchmark questions.

- **Benchmark Initiation:** The evaluation starts with a "benchmark.json" file containing test questions. These questions are extracted and input into the "Ollama LLM benchmark question input". At this point, the workflow splits into two parallel paths for comparative evaluation.

✓ **Path A: Baseline Evaluation (LLM only)**

This path represents the control group, operating without external context.

- The input is directed to the "LLM only: no context" path.
- The "Ollama LLM" generates an answer based solely on its pre-trained knowledge, producing the "LLM only output".

✓ **Path B: Agentic RAG Pipeline**

This path utilizes the pre-processed data from Workflow 1 to augment the generation process.

- **Agentic Planning:** The input is first sent to an "Agentic Planner". This component processes the query and "returns the splitted sub-queries," likely breaking down complex questions into manageable search tasks.
- **Similarity Search:** These sub-queries are fed into the "RAG system: similarity search". As indicated by the connecting arrow, this step queries the "FAISS" vector store created in Workflow 1 to find relevant information.
- **Agentic Retrieval Refinement:** The initial search results are passed to an "Agentic Retrieval" component. This agent filters and refines the results, returning only the "top_k most relevant" documents.
- **Contextualized Generation:** The refined context is provided to the "Ollama LLM," which generates the final "RAG output".

Final Evaluation

The outputs from both paths converge for final assessment.

- **Judge Agent:** Both the "LLM only output" (Path A) and the "RAG output" (Path B) are inputs for the "Judge Agent".
- **Outcome:** The Judge Agent evaluates the outputs and produces a "Final Evaluation" document.

2.2 The Agentic Ingestion Pipeline

A significant limitation in conventional RAG architectures involves the disruption of semantic continuity caused by fixed-size chunking strategies (e.g., arbitrary splitting every 500 characters). This mechanical approach often severs the logical link between a subject and its associated predicates, resulting in fragmented information where a key entity might be separated from its description. To mitigate this, the proposed system integrates a specialized "**Agentic Chunker**" designed to preserve semantic integrity during the ingestion phase.

2.2.1 Semantic Decomposition

Definition of Chunking

In the context of Retrieval-Augmented Generation (RAG), a "chunk" refers to the fundamental unit of text that is indexed, embedded, and retrieved by the system. It represents a specific segment of the original document, ranging from a few words to several paragraphs, that serves as the retrieval target for the vector database. The quality and granularity of these chunks are determinant factors for system performance: chunks that are too large may contain irrelevant noise, while chunks that are too small may lack sufficient context.

Proposed Approach: Atomic Propositions

In contrast to mechanical segmentation based on character or token counts, the system employs the Llama-3:8b model to parse and interpret raw documents from the StatPearls corpus. The model is prompted to decompose complex paragraphs into "atomic propositions", discrete units of information capable of retaining their full meaning in isolation.

Functionally, an atomic proposition is defined as a concise, independent statement containing a single factual assertion. A critical component of this process is coreference resolution. When the original text contains pronominal references (e.g., "it," "he," "they"), the Agentic Chunker identifies the antecedent and rewrites the sentence to explicitly include the specific entity names. This transformation ensures that every indexed chunk is semantically self-contained, eliminating dependencies on surrounding text and improving retrieval precision.

2.2.2 Storage, Embedding and Indexing

Following the semantic decomposition phase, the system implements a dual-storage strategy to decouple content management from vector search operations.

First, the textual content of each atomic proposition is persisted in a **MongoDB** document store. This NoSQL database serves as the ground truth repository, allowing for the retrieval of the original text payload via unique identifiers once a semantic match is located.

Simultaneously, the text is transformed into high-dimensional vector embeddings. For this task, the system utilizes the **all-MiniLM-L6-v2** model from the **HuggingFace Sentence Transformers** library. This specific model was selected for its optimal trade-off between inference speed and embedding quality (semantic density), making it suitable for real-time applications.

These generated embeddings are subsequently indexed in a **FAISS** (Facebook AI Similarity Search) vector database. The index is configured to utilize **cosine similarity** as the distance metric. Unlike Euclidean distance, which measures the magnitude of difference between two points, cosine similarity measures the cosine of the angle between two non-zero vectors in a multi-dimensional space. This approach is particularly effective for high-dimensional semantic data because it focuses on the orientation (i.e., the thematic alignment) of the vectors rather than their magnitude, enabling the efficient calculation of semantic proximity between the query and the stored propositions. It is important to note that the FAISS index remains lightweight by storing only the vector representations and their mapping IDs, which correspond directly to the full text entries residing in the MongoDB collection.

2.3 The Multi-Agent Retrieval Workflow

The proposed system transcends the linear "search-then-answer" paradigm typical of standard RAG implementations. Instead, it adopts a dynamic, Agentic Workflow that actively plans and orchestrates the retrieval process.

In this context, we define an **Agent** not merely as a software script, but as a Large Language Model (LLM) configured to function as a reasoning engine. Unlike a passive model that simply predicts the next word, an agent is capable of perceiving a task, decomposing it into logical steps, using external tools (such as vector databases), and reflecting on its outputs to ensure accuracy.

2.3.1 Query Analysis and Planning

Upon receiving a complex clinical query, the system bypasses immediate retrieval. Instead, the input is first routed to a **Planning Agent**. This component is responsible for **query decomposition**, the process of analyzing the semantic complexity of the user's request and breaking it down into a sequence of targeted sub-tasks.

For example, if a user inquires about *"treatment options and side effects for hypertension,"* the Planner recognizes this as a multi-faceted request. It subsequently generates distinct search intent queries, one specifically for "hypertension treatments" and another for "hypertension side effects." This decomposition strategy ensures that the subsequent retrieval phase comprehensively addresses all dimensions of the user's prompt, preventing information loss.

2.3.2 The Retrieval Module

Once the execution plan is established, the **Retrieval Module** is triggered. This module performs parallel similarity searches within the FAISS vector index for each sub-query generated by the Planner.

The system aggregates the results from these multiple search streams into a unified candidate list. To maintain efficiency and reduce redundancy, a **de-duplication protocol** is applied, filtering out identical document chunks that may have been retrieved by overlapping sub-queries.

2.3.3 Context Re-ranking

A critical distinction in information retrieval is that vector similarity does not always equate to semantic relevance. To address this, the pipeline incorporates a **Re-ranking Step** (often referred to as Agentic Retrieval).

In this phase, an LLM functions as a **discriminator**. It systematically reviews the aggregated candidate documents, comparing them against the original query to assess their actual utility. The model assigns a relevance score to each chunk and filters out noise. **Noise** is defined as documents that share superficial keywords with the query but discuss unrelated medical contexts. This process ensures that the **top-k** documents passed to the final generator are strictly relevant, thereby maximizing the signal-to-noise ratio.

2.3.4 Answer Generation

The final phase of the workflow is **Answer Generation**. The refined context, now stripped of irrelevant data, is synthesized with the original user query and forwarded to the Generator Agent.

To mitigate the risk of **hallucination**, a common failure mode where models fabricate plausible but incorrect facts, this agent operates under strict "grounding" instructions. It is constrained to synthesize answers solely derived from the provided context. Furthermore, the model is explicitly instructed to acknowledge uncertainty and refuse to answer if the retrieved evidence is insufficient, prioritizing clinical safety over conversational fluency.

The specific system prompts utilized for the Chunking, Planning, Retrieval, and Judge agents are provided in the Appendices.

Κεφάλαιο 3. System Evaluation

3.1 Experimental Setup

This section outlines the experimental environment employed for the implementation and evaluation of the Agentic RAG system. It details the hardware specifications, the software stack, and the datasets utilized for the knowledge base and benchmarking processes.

3.1.1 Hardware Environment

All experiments, including data preprocessing, vector embedding generation, and LLM inference, were conducted locally on a personal laptop.

The specific hardware configuration is as follows:

- **Processor (CPU):** AMD Ryzen 7 5800H
- **Memory (RAM):** 16GB DDR4
- **Graphics Unit (GPU):** NVIDIA GeForce RTX 1650 4GB VRAM
- **Operating System:** Windows 11

3.1.2 Software and Models

The core of the system relies on the Ollama framework, which facilitates the local orchestration of Large Language Models. The software stack and models selected for this research include:

1. **LLM Inference**: Ollama 0.15.4
2. **Generative Model:** Llama-3 8B model runs locally, which helps with data privacy and speed.
3. **Document Store:** MongoDB is used to store the textual content of the generated chunks, serving as the system's primary knowledge base.
4. **Vector Database:** FAISS is used for efficient storage and retrieval of the text vectors.
5. **Embeddings:** all-MiniLM-L6-v2 (via HuggingFace) handles the conversion of text into vectors.
6. **Orchestration:** LangChain is used to manage the flow between the different agents and tools.
7. **Programming Language:** Python 3.10.1

3.1.3 Knowledge Base Dataset

The foundation of the system's retrieval capabilities is a specialized medical corpus sourced from the **MedRAG** benchmark suite, specifically utilizing the **StatPearls** dataset. StatPearls is widely recognized as a comprehensive, peer-reviewed repository of clinical summaries, designed primarily for healthcare professionals.

Data Structure and Content

The raw data is serialized in **JSONL** (JSON Lines) format, ensuring efficient line-by-line parsing during the ingestion phase. Each document represents a discrete medical topic, ranging from pharmaceutical agents to complex pathologies and is structured with key-value pairs including unique identifiers (id), article titles (title), and the textual body (content).

Dataset Volume and Sampling

The complete corpus comprises **9,625 JSONL files**, representing a vast breadth of medical knowledge. However, for the purposes of this study, specifically to evaluate the architectural viability of the *Agentic Workflow* within the previously defined local hardware constraints, a representative subset of the **first 100 documents** was utilized. This sampling strategy provides sufficient semantic diversity to benchmark the system's reasoning and retrieval logic without exceeding the memory throughput limits of the experimental setup.

3.1.4 Evaluation Benchmark Dataset

To quantitatively assess the system's reasoning capabilities and measure hallucination rates against a verified standard, we employed a structured evaluation benchmark (benchmark.json). This dataset mimics the format of standardized medical licensing examinations, designed to test not just factual recall but also diagnostic reasoning.

Data Schema and Format

The benchmark is serialized as a large-scale JSON corpus, structured specifically for Multiple-Choice Question Answering tasks. Each entry in the dataset adheres to a strict schema comprising three primary fields:

- **Query Stem** (question): A clinical scenario or direct medical inquiry detailing a patient's presentation or specific pathology.
- **Candidate Options** (options): A set of distractors and the correct answer (typically 4 choices), requiring the model to discriminate between plausible but incorrect medical statements.
- **Ground Truth** (answer): A distinct field containing the verified correct response, serving as the reference standard for automated evaluation.

Sampling Strategy

The full benchmark repository contains a vast array of questions. For the purposes of this study, a **randomized subset of 150 questions** was extracted using a stochastic sampling method. This sample size was calculated to provide a statistically significant indication of system performance (reasoning accuracy and hallucination frequency) while remaining computationally feasible within the latency and throughput constraints of the local hardware environment described in Section 3.1.1.

3.2 Classic *vs* Agentic Chunking

3.2.1 Experiments with Classic chunking parameters

➢ Experiments with **Ollama 3:8b** model

✓ Impact of Chunk Size on RAG Performance

In Figure 2 we can observe how the accuracy of the RAG system fluctuates by varying the **chunk size**.

In this experiment we keep the chunk overlap = 100

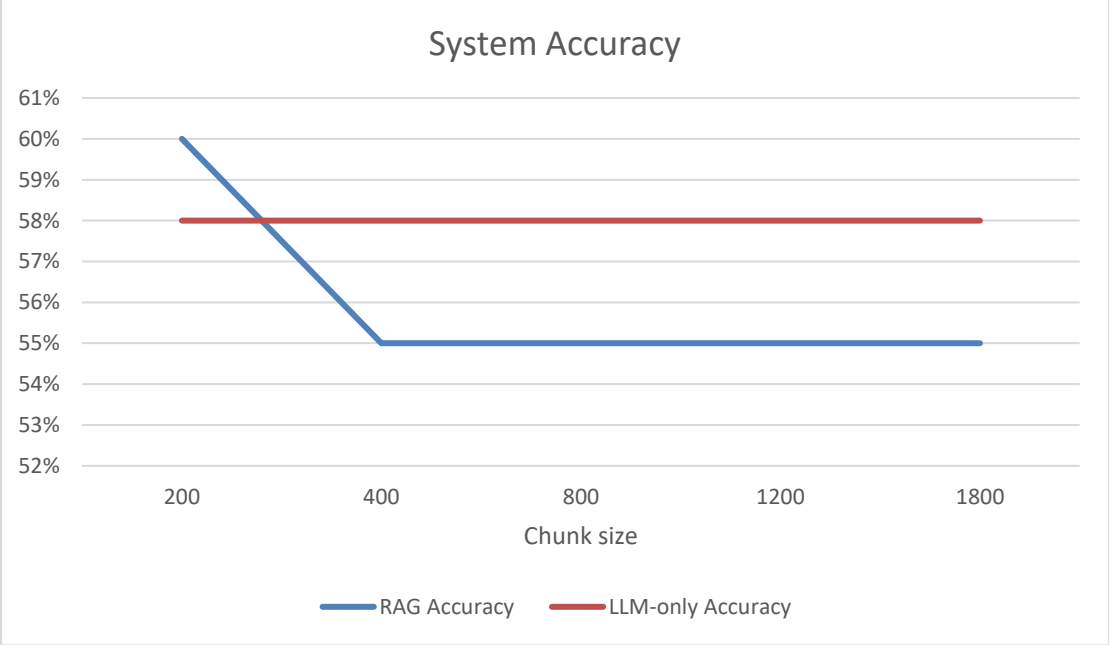

Figure 2. Varying Chunk Size - Ollama 3:8b

Conclusions

The results indicate that system performance varies as a function of chunk size in the RAG configuration, whereas the LLM-only baseline remains stable across conditions, as it operates independently of retrieved context. Specifically, the RAG model achieves its highest accuracy (60%) at the smallest evaluated chunk size (200). However, when the chunk size increases to 400, accuracy declines to 55% and remains at this level for larger chunk sizes (800–1800). In contrast, the LLM-only approach maintains a constant accuracy of 58% regardless of chunk size.

These findings suggest that retrieval granularity has a measurable impact on RAG performance. Smaller chunks appear to provide more precise contextual signals, potentially improving the relevance of retrieved information and supporting better answer generation. As chunk size increases, the informational density within each retrieved unit may introduce additional noise or reduce retrieval specificity, thereby limiting performance.

Importantly, the RAG model outperforms the LLM-only baseline only at the smallest chunk size. For larger chunk sizes, the LLM-only approach yields higher accuracy. This indicates that the benefits of retrieval augmentation are not uniform and depend critically on the segmentation strategy applied to the knowledge base.

Overall, the experiment highlights the sensitivity of retrieval-augmented systems to chunk size selection and underscores the need for careful tuning of retrieval parameters in order to realize performance gains over non-retrieval baselines.

- ✓ Impact of Chunk Overlap on RAG Performance

  In Figure 3 we can observe how the accuracy of the RAG system fluctuates by varying the **chunk overlap**.

  For the chunk overlap experiment, the chunk size was fixed at 800 tokens rather than using the best-performing value identified previously. This choice was made to isolate the effect of overlap while keeping other parameters constant. Using a single, mid-range chunk size avoids confounding effects between chunk size and overlap and allows a clearer interpretation of the overlap parameter independently. A joint optimization of both parameters was beyond the scope of the present study.

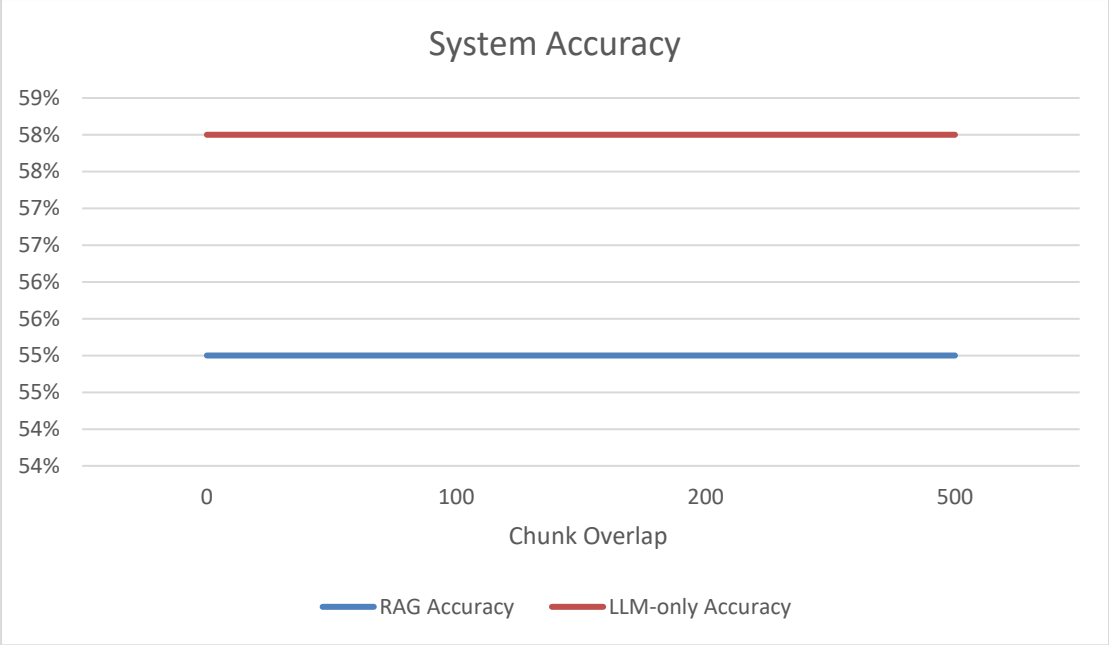

Figure 3. Varying Chunk Overlap – Ollama 3:8b

Conclusions

The results suggest that varying the chunk overlap does not materially influence system performance under the examined conditions. Across all tested overlap values (0–500), the RAG configuration maintains a constant accuracy of 55%, while the LLM-only baseline remains stable at 58%.

This stability indicates that, within the explored range, increasing overlap between adjacent chunks neither improves nor degrades retrieval effectiveness in a measurable way. The absence of variation in RAG performance may imply that the retrieval mechanism is not particularly sensitive to contextual redundancy introduced by overlapping segments, at least for the given dataset and evaluation setup.

Moreover, the LLM-only model consistently outperforms the RAG configuration across all overlap settings. This suggests that, unlike chunk size (as observed in the previous experiment), chunk overlap does not serve as a critical tuning parameter for enhancing retrieval-augmented performance in this case.

Overall, the findings indicate that modifying chunk overlap alone is insufficient to produce performance gains and that other aspects of the retrieval pipeline may play a more decisive role in determining system accuracy

- Experiments with **Ollama 3.2:1b** model
  - ✓ Impact of Chunk Size on RAG Performance

    In Figure 2 we can observe how the accuracy of the RAG system fluctuates by varying the **chunk size**.

    In this experiment we keep the chunk overlap = 100

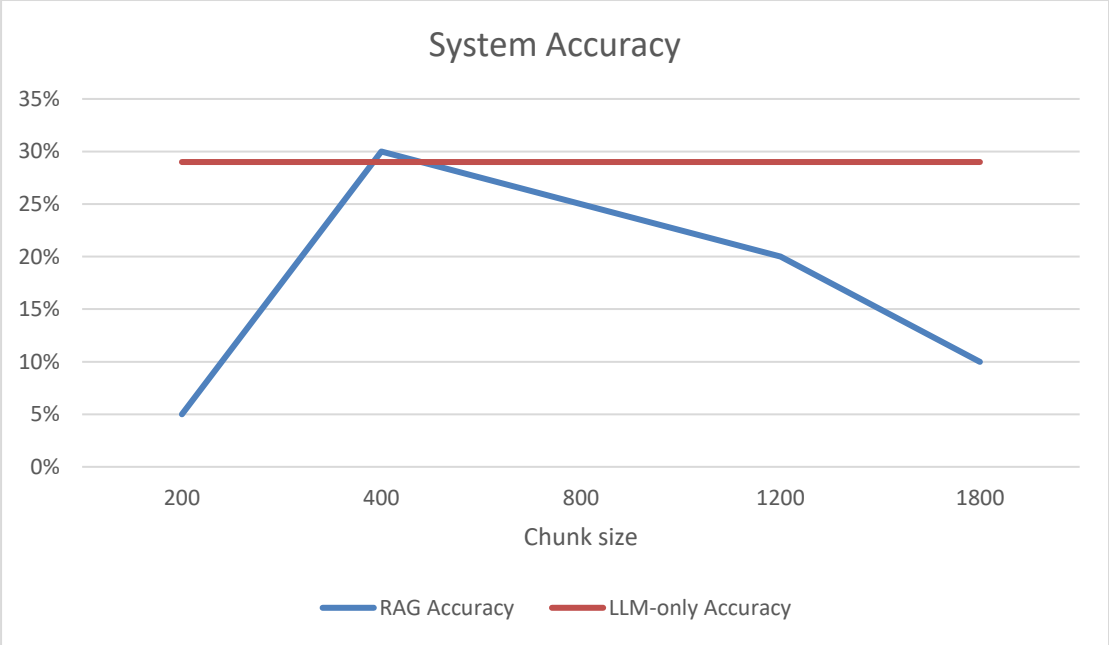

Figure 4. Varying Chunk Size - Ollama3.2:1b

Conclusions

The results obtained with the Ollama 3.2:1b model show a markedly different pattern compared to the previous experiments conducted with Ollama 3:8b. In this setting, **system performance exhibits substantial sensitivity to chunk size, and overall accuracy levels are considerably lower.**

Specifically, the RAG configuration achieves its highest accuracy (approximately 30%) at a chunk size of 400. Performance is substantially lower at 200 (around 5%), improves sharply at 400, and then declines progressively as chunk size increases further (to approximately 20% at 1200 and 10% at 1800). This pattern suggests the existence of an intermediate chunk size that provides a more suitable balance between contextual completeness and retrieval precision for this smaller model.

In contrast, the LLM-only baseline remains stable at approximately 29% across all chunk sizes, as it operates independently of retrieved context. Notably, except for the 400-token condition, where RAG slightly exceeds the baseline, the LLM-only configuration consistently outperforms the retrieval-augmented approach. This indicates that, for the 1b parameter model, retrieval augmentation does not reliably yield improvements and may even introduce degradation when chunk size is not optimally selected.

When compared to the earlier results with the 8b model, two differences are evident. First, **overall accuracy is substantially lower with the 1b model**, reflecting reduced representational and reasoning capacity. Second, the smaller model appears more sensitive to retrieval configuration, particularly chunk size. The retrieval process may impose additional cognitive demands that the smaller model cannot consistently manage, except under carefully tuned conditions.

Overall, the findings suggest that the effectiveness of retrieval augmentation is closely linked to model scale. **For smaller models such as Ollama 3.2:1b, the benefits of RAG are limited and highly dependent on parameter selection, whereas larger models appear more robust to retrieval-related design choices.**

✓ Impact of Chunk Overlap on RAG Performance

In Figure 3 we can observe how the accuracy of the RAG system fluctuates by varying the **chunk overlap**.

For the chunk overlap experiment, the chunk size was fixed at 800 tokens rather than using the best-performing value identified previously. This choice was made to isolate the effect of overlap while keeping other parameters constant. Using a single, mid-range chunk size avoids confounding effects between chunk size and overlap and allows a clearer interpretation of the overlap parameter independently. A joint optimization of both parameters was beyond the scope of the present study.

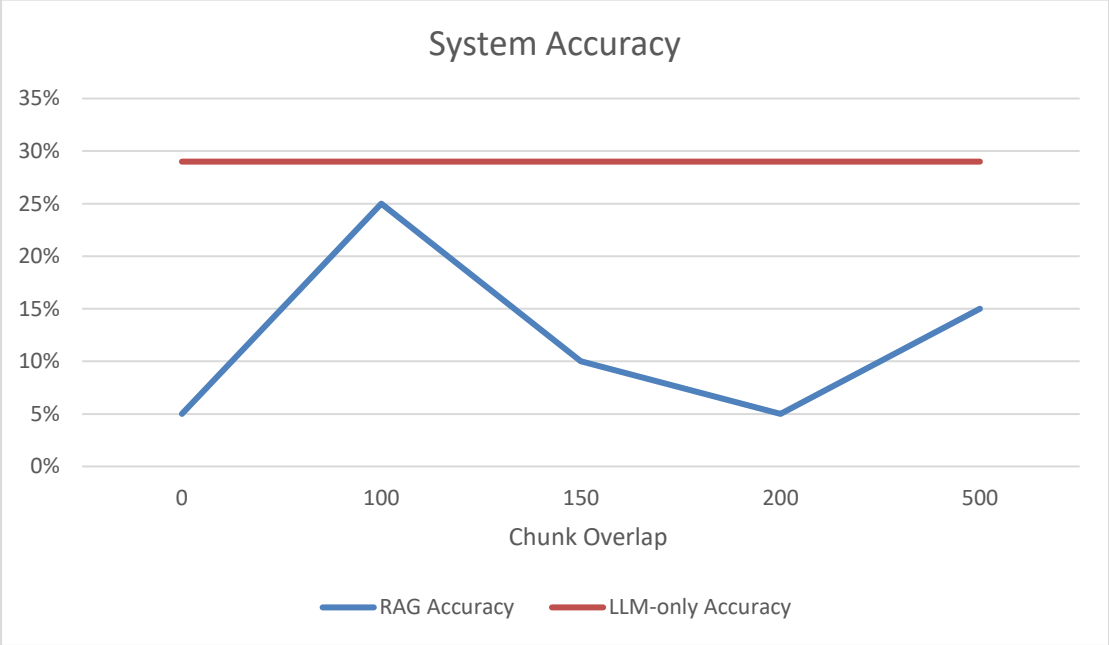

Figure 5. Varying Chunk Overlap - Ollama3.2:1b

Conclusions

The results for the Ollama 3.2:1b configuration indicate that chunk overlap has a noticeable impact on the performance of the RAG system, although this impact is not consistently positive. RAG accuracy varies substantially across the tested overlap values (0–500), reaching its highest value at an overlap of 100, and declining at intermediate values (150 and 200) before partially recovering at 500. This pattern suggests that moderate overlap may facilitate improved contextual continuity during retrieval, whereas higher overlap levels may introduce redundancy or reduce retrieval precision.

Despite this variability, the LLM-only baseline remains consistently superior across all configurations. Its performance is stable and unaffected by chunk overlap, maintaining a clear margin over the RAG system even at the latter's best-performing setting. Notably, even at the optimal overlap value (100), the RAG configuration does not exceed the baseline accuracy.

These findings indicate that, **for the smaller 1b model, chunk overlap constitutes a sensitive parameter within the retrieval-augmented pipeline, influencing outcomes more visibly than in larger models.** However, tuning overlap alone is insufficient to bridge the performance gap between RAG and the standalone LLM. Overall, the results suggest that additional modifications to the retrieval strategy, indexing approach, or prompt formulation may be necessary to realize tangible gains in performance for this model size.

Overall Conclusions and Model Comparison

Across all four experiments, **the smaller model (Ollama 3.2:1b) consistently demonstrates lower performance compared to the larger model (Ollama 3:8b)**, regardless of configuration. The performance gap remains evident in both LLM-only and RAG settings and is not eliminated through parameter tuning.

While the larger model is able to benefit from retrieval augmentation under certain chunk size configurations, the smaller model shows limited and inconsistent gains from RAG. In several cases, retrieval either fails to improve performance or introduces additional variability. This indicates that the smaller model is more sensitive to retrieval design choices and less capable of effectively integrating retrieved context into its responses.

Overall, the results suggest that model capacity plays a central role in determining the effectiveness of retrieval augmentation. For smaller models, performance appears to be primarily constrained by limited representational and reasoning ability, and adjustments to chunk size or overlap alone are insufficient to produce substantial improvements.

3.2.2 Experiments with Agentic Chunking

In Agentic Chunking, we still use a text splitter (RecursiveCharacterTextSplitter) to break down very large articles so they fit within the agent's context window. Additionally, using the splitter prevents the agent from being overwhelmed by too much information and from hallucinating. The following experiment examines the use of the pre-splitter before providing the large medical articles to the agent.

The semantic segmentation process is orchestrated by the Llama-3-8B-Instruct model, deployed via the Ollama runtime environment. This specific architecture was selected based on a strategic evaluation of the trade-off between reasoning capability and computational throughput

Table 1. Pre-splitter in Agentic chunking with Ollama 3:8b

	RAG Accuracy
With pre-splitter	54%
Without pre-splitter	52%

In the pre-splitter we keep the parameters below:

Chunk size=2000

Chunk overlap=100

Justification for the parameters

The 2000-token window was selected to capture adequate semantic context, ensuring that long-range dependencies (e.g., a pronoun referring to a subject mentioned paragraphs earlier) are preserved for the agent to resolve. A smaller window could sever these linguistic links, degrading the reasoning quality.

The 100-token overlap acts as a continuity safeguard, preventing the truncation of sentences at chunk boundaries and maintaining the logical flow between segments.

Comparison between the 2 models (with use of Agentic Chunking)

This comparison evaluates the RAG & LLM-only agents, distinct from the Agentic Chunking.

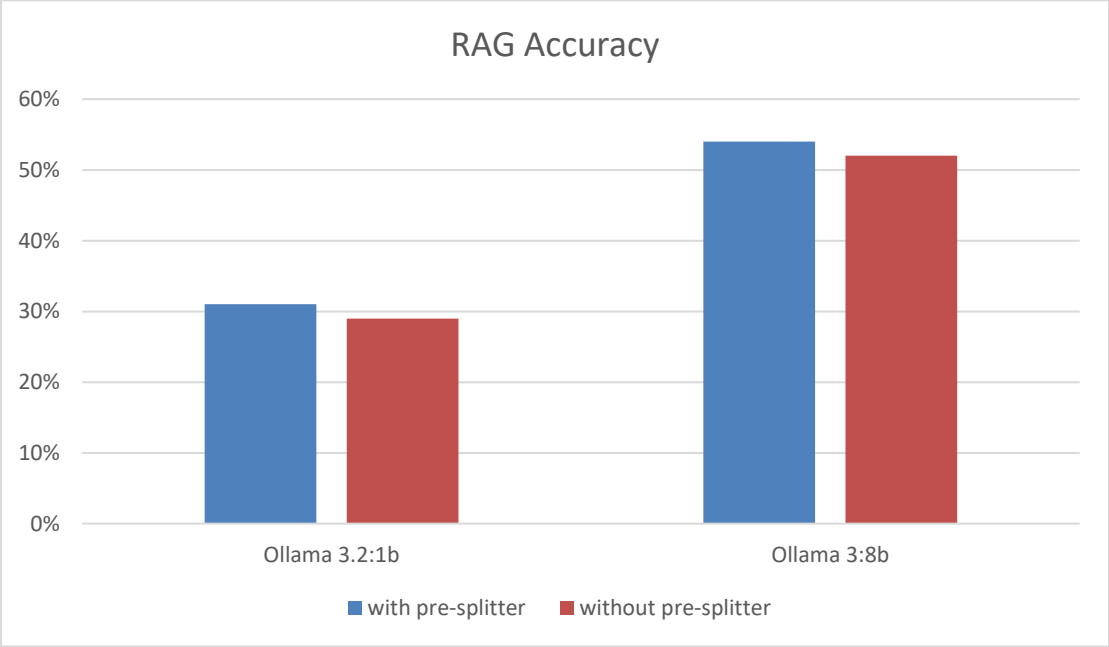

Figure 6. Agentic Chunking-Comparison between the 2 models

Conclusions

The results presented in the chart highlight the impact of model size and pre-processing configurations on system accuracy during agentic chunking. Two primary observations can be drawn from the data:

- **Impact of Model Capacity:** There is a distinct performance disparity between the two models. The Ollama 3:8b model significantly outperforms the smaller Ollama 3.2:1b variant. While the 1b model hovers around 30% accuracy, the 8b model achieves accuracy rates in the mid-50s range. This indicates that the underlying model's reasoning capability is the dominant factor in the system's overall performance, regardless of the chunking configuration used.

- **Effect of the Pre-splitter:** The inclusion of a pre-splitter yields a consistent but marginal improvement across both models. For the 1b model, accuracy increases from approximately 29% to 31%, and a similar gain is observed in the 8b model (rising from roughly 52% to 54%). This suggests that while the pre-splitter serves as a valid optimization for the agentic chunking pipeline, it acts as an incremental enhancement rather than a transformative factor compared to the model size itself.

3.2.3 Comparison Classic *vs* Agentic Chunking

To ensure a fair and rigorous evaluation, we benchmarked the **Agentic Chunking** pipeline against the **optimal configuration** of the **Classic Chunking** method (chunk_size =200, chunk_overlap=100), which was empirically determined to yield the highest baseline accuracy.

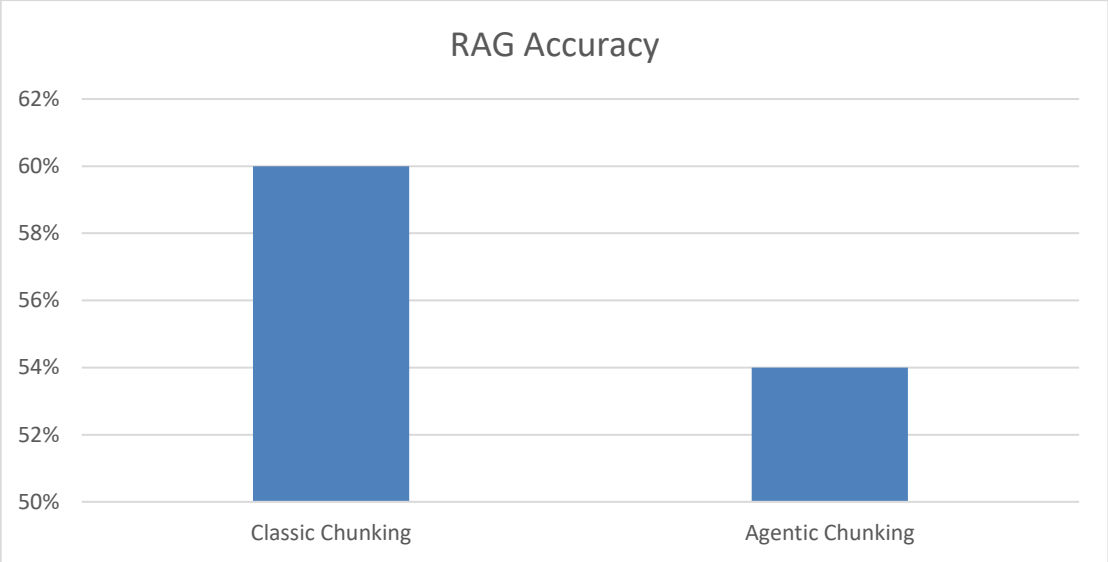

Figure 7. Classic vs Agentic chunking

Conclusions

1. The "Semantic Overshoot" Effect

Contrary to the hypothesis, Optimized Classic Chunking outperformed the Agentic Pipeline (60% vs. 54%). This 6% disparity is attributed to "Semantic Overshoot": the agentic decomposition was so rigorous in isolating facts that **it inadvertently stripped away peripheral context**. More specifically, keywords that, while not grammatically essential, acted as valuable associative anchors for the vector search mechanism.

2. Configuration Overhead

While the Agentic approach suffered from the reasoning limitations of the Llama-3-8B model, it offers a critical operational advantage: **Parameter Autonomy**. Unlike the Classic method, which required extensive manual tuning to reach its peak, the Agentic pipeline functions without hyperparameter optimization. Thus, the slight reduction in accuracy represents the cost of achieving a fully automated, maintenance-free ingestion workflow.

3.3 Agentic Retrieval and Planner

3.3.1 Performance Analysis of Agentic Retrieval

All the calculations in the experiments below were performed using the Agentic Chunking (with the use of pre-splitter) approach.

The Agentic Retrieval process is orchestrated by the Llama-3-8B-Instruct model, deployed via the Ollama runtime environment.

- Experiment using Ollama 3.2:1b model for the RAG and LLM-only Agents

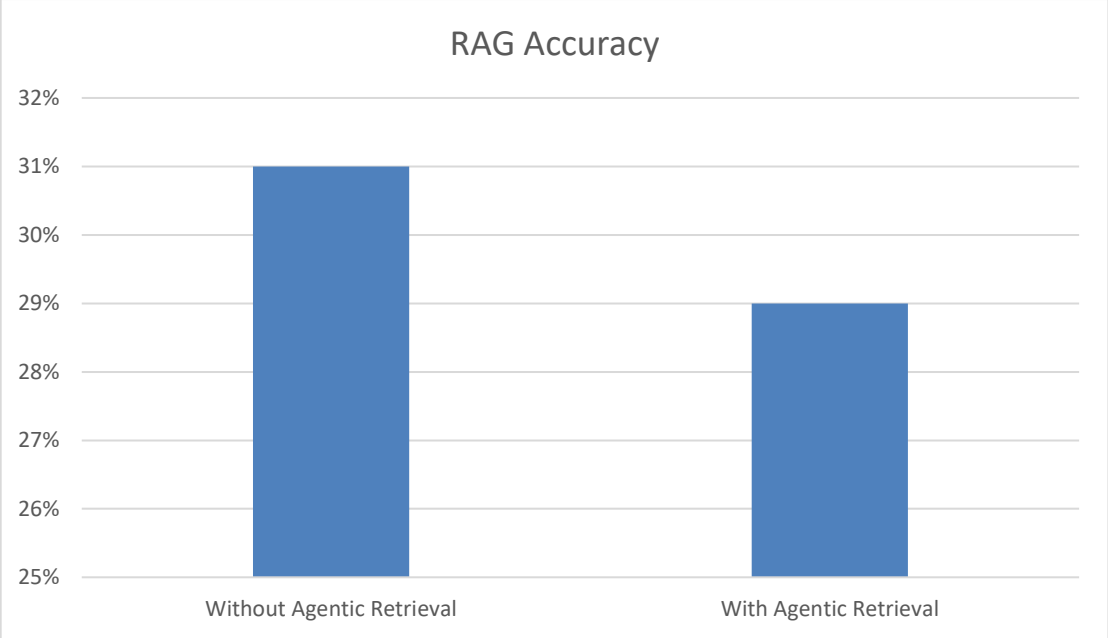

Figure 8. Impact of Agentic Retrieval in Ollama 3.2:1b

Conclusions

The graph clearly demonstrates that for the Ollama 3.2:1b model, **implementing Agentic Retrieval actually hinders performance**, causing accuracy to drop from 31% to 29%. This suggests that the 1-billion parameter model lacks the sufficient reasoning capabilities required to handle the complex decision-making and multi-step logic of agentic workflows, leading to confusion rather than better results.

➢ Experiment using Ollama 3:8b model for the RAG and LLM-only Agents

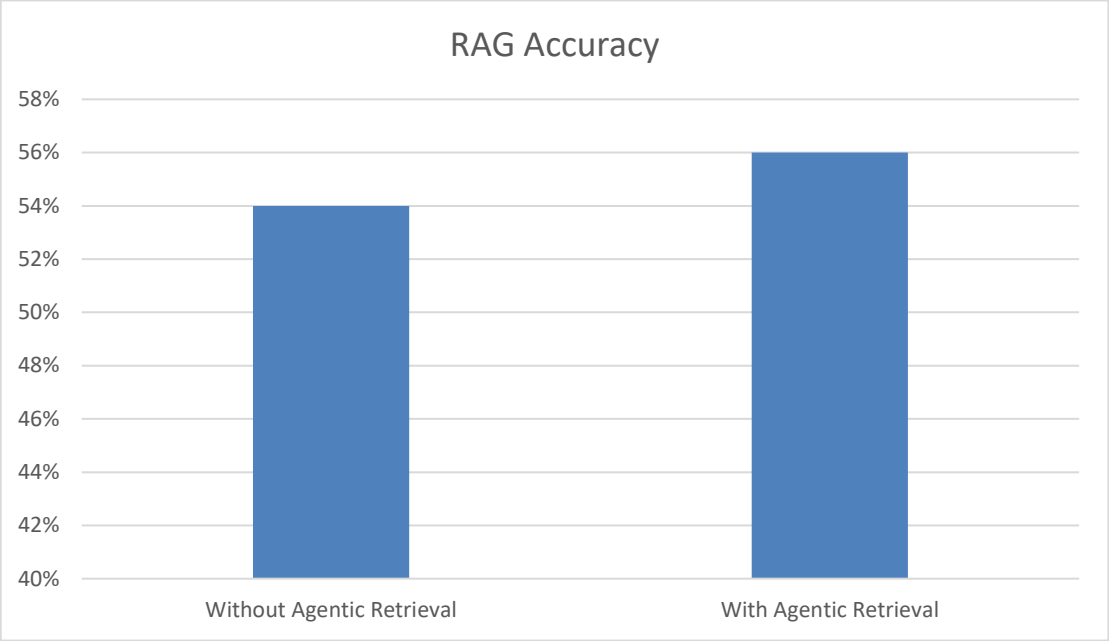

Figure 9. Impact of Agentic Retrieval in Ollama 3:8b

Conclusions

Based on the provided chart for the Ollama 3:8b model, **the implementation of Agentic Retrieval leads to a clear performance improvement**, increasing accuracy from 54% to 56%. This contrasts sharply with smaller models, suggesting that the 8-billion parameter architecture possesses the necessary reasoning depth to handle the complexity of agentic workflows. Rather than being confused by the additional decision-making steps, the model effectively leverages the agentic capabilities to retrieve better context and refine its answers.

However, while the 2% gain confirms that the model is "agent-ready," it is worth noting that this improvement likely comes with increased latency due to the multi-step nature of agentic loops. Since the accuracy gain is positive but modest, the decision to use Agentic Retrieval with this model should depend on the specific use case: it is a strong choice for complex queries where precision is paramount, but a standard RAG pipeline might still be preferable for applications requiring real-time speed.

Overall Conclusions and Model Comparison

Comparing the two models reveals a distinct **"reasoning threshold"** required for Agentic Retrieval to be effective. While the lightweight **1B model** suffered a performance regression (dropping from 31% to 29%) because it was overwhelmed by the added complexity, the more capable **8B model** successfully leveraged the agentic workflow to boost accuracy from 54% to 56%.

This suggests that Agentic Retrieval is not a universal solution but rather a **capability multiplier** that depends heavily on the underlying model's intelligence. Small models tend to get confused by the decision-making overhead of agents, whereas mid-sized models like the 8B possess sufficient reasoning depth to navigate these multi-step processes and actually extract value from them.

3.3.2 Performance Analysis of the Planner

The process of the Planner is orchestrated by the Llama-3-8B-Instruct model, deployed via the Ollama runtime environment.

All the calculations in the experiments below were performed using the Agentic Chunking (with the use of pre-splitter) and Agentic Retrieval approach.

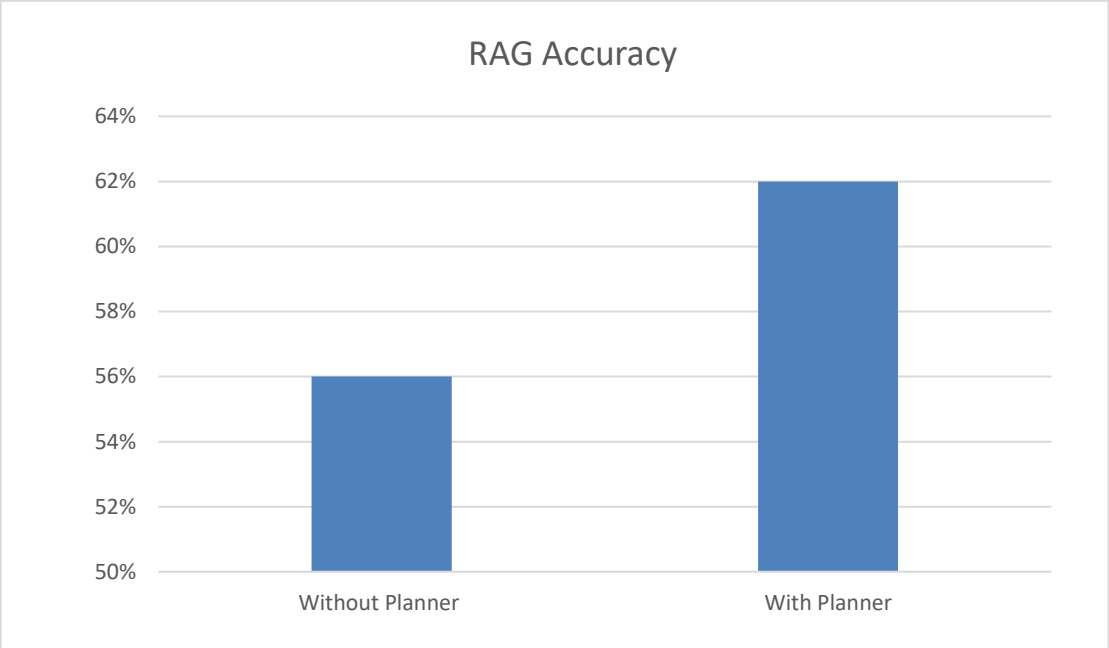

Figure 10. Impact of Planner

Conclusions for the RAG Accuracy

Based on the graph provided for the Agentic Planner, **the inclusion of a planning step results in a remarkable improvement in performance**, boosting RAG accuracy from 56% to 62%. This 6% increase is a significant leap, suggesting that the primary challenge was not just accessing information, but effectively structuring the approach to the problem before attempting to solve it.

This indicates that adding a "Planner" module is highly effective for this architecture. By breaking down complex queries into a logical sequence of steps (decomposing the problem) before retrieving data, the system avoids getting lost in irrelevant details. Essentially, the data proves that a structured "think before you act" mechanism adds much more value here than simple retrieval enhancements alone.

Conclusions for the efficiency of the Planner

Based on the quantitative analysis of the retrieval logs from 150 benchmark questions and 445 retrieved document chunks, the following critical conclusions regarding the Planner module's efficacy were drawn:

1. Expansion of Retrieval Scope

The integration of the Planner module drastically improved the system's recall capabilities. Specifically, the **Planner successfully identified 71.24%** of all relevant documents, whereas the standalone original query **(Baseline) retrieved only 26.52%**.

2. Bridging the Semantic Gap

The most significant finding of this experiment is the Planner's **exclusive contribution**. A total of **52.81%** of the retrieved knowledge, more than half of the relevant context, was discovered **exclusively through the sub-queries** generated by the Planner. These documents would have been completely missed by the baseline method.

The Planner effectively acts as a semantic bridge, translating vague user intent into precise technical queries, thereby eliminating the "blind spots" of standard retrieval.

3. Superior Quality of Unique Retrievals

Beyond quantity, the Planner also demonstrated superior quality in its unique discoveries. The average *relevance match score* for documents found *only* by the Planner was 1.60, compared to 1.00 for documents found *only* by the original query.

The *Relevance Match Score* simply measures how many different search queries found the same document.

When the system searches, it doesn't just ask the original question once, but it creates several different versions of that question (sub-queries). The Match Score counts how many of these different versions pointed to the exact same piece of information.

- A score of 1.00 means the document was found by only one search phrase.
- A higher score (e.g., 1.60) means the document was found by multiple different search phrases simultaneously.

Essentially, the higher the score, the more 'agreed upon' the document is, proving it is a central and reliable answer to the user's problem..

The additional context provided by the Planner is not merely "noise" but **consists of high-value information that is semantically closer to the ground truth** than the incidental hits of the baseline search.

4. System Synergy and Validation

Approximately **18.43%** of the documents were retrieved by both methods (overlap). These documents exhibited the highest average match score (**2.85**), representing the core knowledge base. Consequently, the Planner serves a dual role: it uncovers new, hard-to-find information while simultaneously reinforcing the system's confidence in the most obvious evidence.

Summary of the Planner's efficiency

Table 2 and Figure 11 present a quantitative summary of the Planner's efficiency, based on the dataset of 150 questions used in our experiments.

Table 2. Summary of the Planner efficiency

Retrieval Method	Documents Found	Contribution (%)
Total Retrieved	**445**	**100%**
Found by Original Query	118	26.52%
Found by Planner	317	71.24%
Exclusive to Planner	**235**	**52.81%**
Overlap (Both)	82	18.43%

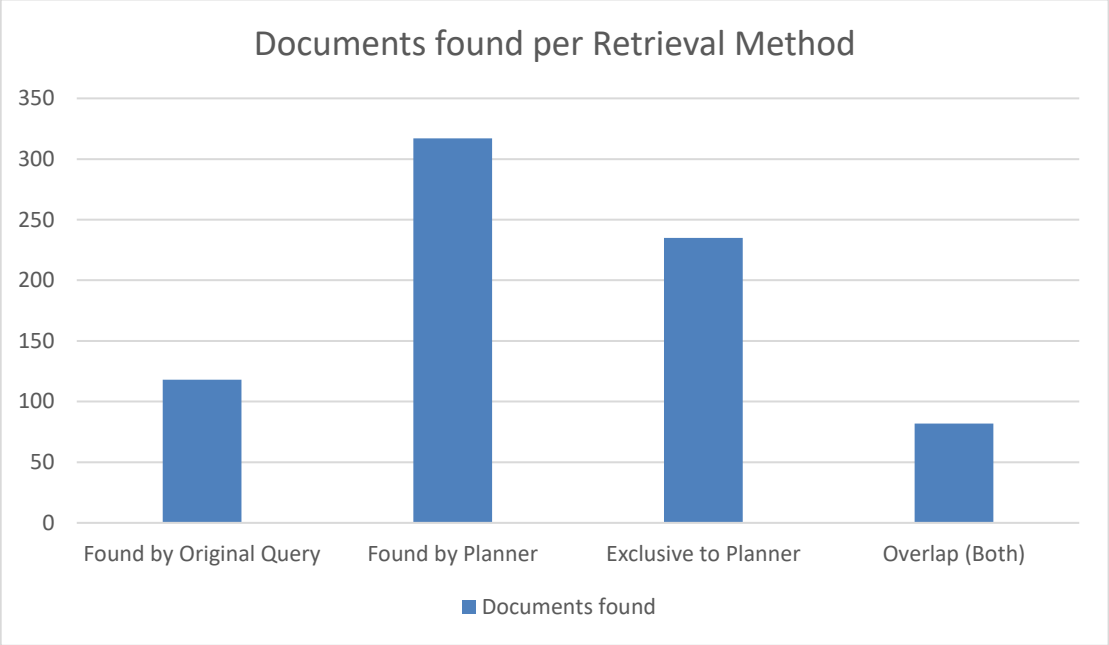

Figure 11. Chart of the Planner efficiency per Retrieval Method

Conclusions

The data indicates a significant disparity in efficacy between the two retrieval strategies, with the **Planner method demonstrating a far superior capability for document identification.** The Planner successfully retrieved 317 documents, accounting for 71.24% of the total dataset, whereas the Original Query contributed a much smaller proportion at 26.52%. This substantial volume gap suggests that the Planner serves as the primary driver of the retrieval process, while the Original Query functions more as a supplementary or subset method rather than a comprehensive standalone tool.

Furthermore, the analysis of exclusive data highlights the critical importance of the Planner mechanism in broadening the search scope. Over half of the total documents found (52.81%) were exclusive to the Planner, meaning this information would have been entirely missed if relying solely on the Original Query. In contrast, the overlap between the methods (18.43%) indicates that while there is some intersection, the Original Query offers minimal unique coverage, as the majority of its results are already captured within the Planner's broader retrieval set.

3.4 Judge as an Agent

The process of the Judge is orchestrated by the Llama-3-8B-Instruct model, deployed via the Ollama runtime environment.

All the calculations in the experiments below were performed using the Agentic Chunking (with the use of pre-splitter), Agentic Retrieval and Planner approach.

Based on the quantitative analysis of the evaluation logs from 150 benchmark questions, we derived the following critical conclusions regarding the performance and reliability of the "Judge as an Agent" framework:

1. Preference Distribution

The evaluation results indicate a highly competitive baseline, with the Judge agent demonstrating a **slight preference for the No-Context LLM** (54.7%) **over the RAG pipeline** (45.3%). This distribution suggests that the mere injection of external documents does not inherently guarantee a superior qualitative outcome under the Judge's strict criteria. In many scenarios, the baseline LLM leverages its robust internal parametric memory to generate highly fluent, coherent, and direct medical justifications.

In this evaluation, 'preference' is defined as the binary outcome of the Judge Agent's comparative assessment for each individual question. For every test case, the Judge reviews both the Baseline (No-Context) answer and the RAG answer side-by-step, assigns a reasoning score (1-10) to each, and explicitly declares a 'Winner'. Therefore, the 54.7% preference rate signifies that in the majority of head-to-head comparisons, the Judge determined that the Baseline model's reasoning was more logically sound, factual, or coherent than the RAG system's output, regardless of whether the final multiple-choice selection was correct.

Conversely, when the RAG system forces the generator to integrate external chunks, it introduces the risk of contextual interference. If a retrieved document contains redundant or tangentially related clinical data, the model may struggle to synthesize it gracefully, resulting in a clunkier or less focused justification. Consequently, the Judge's preference for the baseline model frequently highlights instances where the model's innate pre-training was fully sufficient, rendering the RAG context as detrimental "cognitive noise" rather than actionable evidence.

2. Scoring Similarity

In terms of quantitative scoring, the **performance gap between the models is minimal**. The RAG system achieved an average score of 8.09/10, whereas the LLM averaged 7.96/10. These figures demonstrate that the Judge considered both models to produce high-quality responses, giving only a marginal advantage to the RAG system.

The *Reasoning Quality Score* is defined as a scalar metric (ranging from 1 to 10) assigned by an independent 'Judge Agent', specifically, a separate instance of the Llama-3-8B model prompted to act as an impartial medical evaluator. Unlike binary accuracy, this score assesses the **logical validity** of the model's justification rather than just the correctness of the final option. The evaluation criteria explicitly weigh **factuality** (alignment with established medical truth), **coherence** (step-by-step logical flow), and **hallucination avoidance**. A score approaching 10 indicates a flawless, clinically sound deduction, while lower scores reflect logical gaps or the misuse of retrieved context.

3. Observation of Context Bias (Crucial Finding)

A significant anomaly was observed in the cases labeled as 'RAG_WORSENED'. In these specific instances, where the RAG system objectively provided a worse answer than the baseline, the Judge still declared the RAG system the winner 66% of the time (18 out of 27 cases). This indicates a 'Context Bias', where **the Judge tends to reward the presence of retrieved information, even when that information reduces the factual accuracy or clarity of the answer.**

The classification labels **RAG_IMPROVED** and **RAG_WORSENED** categorize the net impact of the retrieval pipeline on answer accuracy relative to the ground truth. Specifically, **RAG_IMPROVED** designates instances where the baseline LLM failed to answer correctly, but the inclusion of retrieved context enabled a correct response, highlighting the system's ability to bridge knowledge gaps. In contrast, **RAG_WORSENED** represents a performance regression where the standalone model answered correctly using its internal parametric knowledge, but the RAG system, likely influenced by **"contextual noise"** or irrelevant documents, deviated to an incorrect option.

4. Fairness in Tie Scenarios

When both models answered correctly ('BOTH_CORRECT'), the Judge displayed impartial behavior. The decisions were almost evenly split, with 35 wins for the LLM and 33 for the RAG system. This confirms that **the Judge does not exhibit a strong inherent preference for one model when the factual quality is equal.**

The **BOTH_CORRECT** classification identifies instances of **algorithmic consensus**, where both the standalone LLM and the Agentic RAG system independently converged on the correct answer. This overlap suggests that the query pertained to **foundational medical knowledge**, facts sufficiently represented within the model's pre-trained weights, which the retrieval pipeline successfully reinforced without introducing contradictory noise. In these cases, the primary value of the RAG system shifts from correction to **verification**, providing an auditable trail of evidence (retrieved documents) that substantiates the model's intrinsic reasoning.

3.5 Conclusions

In this chapter, we presented a comprehensive experimental evaluation of the proposed Agentic RAG architecture, assessing the individual impact of each module, from data ingestion to retrieval and automated judgment. The experimental results lead to three key conclusions regarding the system's performance and behavior in the medical domain.

First, regarding **Data Ingestion**, our analysis demonstrates that **Agentic Chunking** (enhanced by a pre-splitting mechanism) offers a robust alternative to classic static chunking. While the overall retrieval accuracy remains comparable to traditional methods, the agentic approach exhibits superior stability and effectiveness in handling complex, unstructured medical texts, ensuring that semantically complete propositions are indexed.

Second, the **Agentic Planner** proved to be the most critical component of the pipeline. Quantitative analysis revealed that the Planner is responsible for retrieving **71.24%** of the relevant documents, with **52.81%** of the knowledge being exclusively accessible through its generated sub-queries. This confirms that a dynamic, multi-step planning phase effectively bridges the semantic gap between vague user queries and technical medical terminology, significantly outperforming the single-query baseline.

Third, the **Agentic Retrieval** phase, specifically the automated re-ranking mechanism, proved essential for ensuring context precision. By filtering the broad pool of documents initially gathered by the Planner, this module effectively reduced "context noise" before it reached the generation stage. This filtration process is critical for Small Language Models, which experimental data shows are disproportionately sensitive to irrelevant information. By discarding low-relevance chunks, the Agentic Retriever minimized the cognitive load on the generation model, ensuring that answers were grounded only in the most semantically aligned clinical evidence.

Finally, the deployment of the **Judge Agent** for automated evaluation highlighted a crucial behavioral nuance. While the Judge successfully automated the benchmarking process, it exhibited a **"Context Bias,"** showing a tendency to favor RAG-generated answers (66% preference in error cases) simply due to the presence of retrieved context, even when the factual accuracy was compromised. This finding suggests that while LLM-based judges are powerful, they require careful calibration to distinguish between "richness of information" and "factual correctness."

Collectively, these experiments validate that the shift from rigid, linear RAG workflows to autonomous **Agentic Workflows** provides tangible benefits in recall, semantic relevance, and reasoning depth, establishing a solid foundation for the system's deployment in real-world clinical scenarios.

Κεφάλαιο 4. Discussion and Future Work

4.1 General Observations on Agentic Behavior

The transition from a static, linear Retrieval-Augmented Generation (RAG) architecture to an **Agentic Workflow** marks a significant shift in how the system interacts with medical knowledge. Qualitative and quantitative observations from the experimental phase highlight that the introduction of autonomous agents, specifically the **Planner** and the **Retriever**, fundamentally alters the retrieval dynamics.

The most important impact was how the Agentic Planner connected the user's intent with the actual medical terms in the database. Unlike standard systems that just look for exact word matches, the Planner showed it could break down complex clinical questions into specific searches. This approach allowed the system to find 52.81% of the relevant information that the basic search completely missed.

Moreover, the system proved to be very adaptable. Even when the user's question was unclear or poorly phrased, the Planner was able to rewrite it using the correct medical terms, which led to accurate search results. This shows that the system does not just blindly search for data; it actually understands what the user is looking for, acting as a bridge between the user and the database.

4.2 Assessment of the Evaluation Framework

A key part of this research was using a Judge Agent (powered by Llama-3) to automatically grade the reasoning quality. While the Judge allowed us to evaluate the system at scale, a closer look at its decisions revealed some significant biases that need to be taken into account.

4.2.1 The phenomenon of Context Bias

The most critical finding regarding the automated evaluator is the presence of a strong "Context Bias." The Judge demonstrated a systematic tendency to favor responses that included retrieved context and technical terminology, even when that information was factually incorrect.

The results from the 'RAG_WORSENED' cases, where the RAG system gave an incorrect answer compared to the baseline, revealed a surprising trend. The Judge still picked the RAG system as the 'winner' in 66% of these instances (18 out of 27). This indicates that the evaluator often confuses detail with accuracy. It tends to penalize the short, correct answers of the baseline model, favoring the RAG system simply because its answers sound plausible, even when they are factually wrong.

4.2.2 Blindness to Improvement

On the other hand, the Judge found it difficult to recognize real improvements. In the 'RAG_IMPROVED' cases, where the RAG system successfully fixed a mistake made by the baseline, the Judge only picked the RAG model as the winner 25% of the time (2 out of 8). Essentially, the Judge's own internal knowledge seems to match the mistakes of the baseline model, making it resistant to accepting the correct information found by the system.

4.2.3 Consensus and Fairness

Despite these biases, the Judge displayed impartiality in consensus scenarios. When both models answered correctly (BOTH_CORRECT), the preference distribution was nearly symmetrical (54.7% for Baseline vs. 45.3% for RAG). This confirms that in the absence of conflicting facts, the Judge does not exhibit an inherent architectural preference, validating its utility as a comparative tool for non-controversial queries.

4.3 System Limitations

While the Agentic RAG architecture demonstrates superior recall and reasoning capabilities, it is subject to specific technical and operational limitations:

1. **Latency and Computational Cost:** The agentic workflow inherently introduces latency. The multi-step process requires significantly more inference calls than a standard RAG pipeline. This makes the current implementation less suitable for real-time applications where millisecond-level response times are critical.

2. **Error Propagation:** The system relies heavily on the initial success of the Planner. If the Planner hallucinates irrelevant sub-queries or fails to grasp the core medical concept, this error propagates downstream, leading the Retriever to fetch irrelevant documents.

3. **Dependence on Automated Grading:** As the Judge's bias clearly shows, AI evaluation cannot fully replace human review. Since we optimize the system based on this imperfect judge, there is a risk that we might accidentally encourage the system to give long, detailed answers, even if they are wrong, rather than focusing on being factually precise.

4.4 Future Work

This thesis proves that Agentic RAG is a viable solution for healthcare. However, it also highlights the need for future work to overcome current limitations and expand the system's capabilities.

- **Large-Scale Benchmarking:** The current evaluation was conducted on a pilot dataset of 150 questions. Future work will focus on scaling this to thousands of queries from established medical benchmarks. This will provide the statistical power needed to fully validate the system's robustness across diverse medical specialties.

- **Advanced Planning Mechanisms:** To mitigate error propagation, future iterations will integrate advanced reasoning techniques such as **Chain-of-Thought (CoT)** and **Tree-of-Thoughts (ToT)** into the Planner. This will allow the agent to self-correct its search strategy dynamically before executing queries.

- **Hybrid Evaluation Framework:** To fix the Judge's 'Context Bias,' we plan to build a combined evaluation system. We will train a smaller, focused 'Critic Model' using errors that humans have explicitly flagged. This model will be specifically taught to catch and penalize any made-up information.

- **Multimodal Integration:** Given the visual nature of medical diagnosis, future research will explore extending the RAG pipeline to support **multimodal retrieval**, allowing the system to process and reason over medical imaging (X-rays, MRIs) alongside textual data.

Appendix A: System Prompts

A1 - Agentic Chunker Prompt:

You are a text splitting robot. You *must* follow these rules:

1. Your task is to split the text into a list of self-contained, semantically complete propositions.

2. A proposition is a single, complete statement or idea.

3. You *must* output *only* a valid JSON list of strings.

4. Do NOT under any circumstances output any other text, preamble, conversational reply, or explanation.

5. If you cannot process the text, output an empty JSON list: []

A2 - LLM Baseline model (No-context) Prompt:

You are a helpful medical AI. Answer the user's multiple-choice question and provide a justification for your choice.

A3 - Planner Agent Prompt:

You are an expert researcher. Break down the user's complex medical question into simple keyword search queries.

A4 - Agentic Retrieval Prompt:

You are a retriever agent. Your task is to find the {top_k} most relevant document chunks to the user's question from the provided list of documents.

A5 - RAG Prompt:

You are an expert medical AI utilizing retrieval-augmented generation.

TASK:

Answer the multiple-choice question using the provided Context Documents AND your own expert medical knowledge.

CRITICAL INSTRUCTION - RELEVANCE CHECK:

Before answering, evaluate if the Context Documents are ACTUALLY related to the specific medical topic of the question.

- IF Context is Irrelevant (e.g., talks about "Physical Therapy" while the question is about "Dentistry"): **DISCARD the Context completely** and answer solely based on your internal knowledge.

- IF Context is Relevant: Use it to support your answer and clarify specific details.

INSTRUCTIONS:

1. ANALYZE: Read the context documents carefully.

2. CHECK RELEVANCE: Explicitly ask yourself: "Do these documents discuss the exact same pathology/condition as the question?"

3. SYNTHESIZE: Combine valid context facts with your internal medical expertise.

4. THINK STEP-BY-STEP: Compare the medical facts against options A, B, C, and D.

5. ELIMINATE: Rule out options that are incorrect based on the valid evidence.

6. FORCE SELECTION: You MUST select the most plausible option (A, B, C, or D).

  - Do NOT answer "Not Applicable".

  - Do NOT answer "None".

  - If unsure, pick the best educated guess based on general medical principles.

OUTPUT FORMAT:

You must return a valid JSON object strictly following this schema:

{format_instructions}

A6 – Judge Agent Prompt:

You are an impartial Medical Expert Evaluator (Judge).

TASK:

Compare the reasoning (justification) of two AI models answering a medical question.

DATA:

- Question: {question}

- Correct Answer: {correct_opt}

- Model A (No Context): Answered {llm_ans}.

  Reasoning: "{llm_just}"

- Model B (RAG + Context): Answered {rag_ans}.

  Reasoning: "{rag_just}"

CRITERIA:

1. FACTUALITY: Does the reasoning align with the correct medical answer?

2. COHERENCE: Is the logic sound and step-by-step?

3. HALLUCINATION CHECK: Did Model B force irrelevant context into the answer?

OUTPUT:

Return a JSON with:

- Scores (1-10) for both.

- Winner ("RAG", "LLM", or "TIE").

- A short critique explaining the verdict.

Appendix B: Benchmark Sample

"medqa": {

"0000": {

"question": "A junior orthopaedic surgery resident is completing a carpal tunnel repair with the department chairman as the attending physician. During the case, the resident inadvertently cuts a flexor tendon. The tendon is repaired without complication. The attending tells the resident that the patient will do fine, and there is no need to report this minor complication that will not harm the patient, as he does not want to make the patient worry unnecessarily. He tells the resident to leave this complication out of the operative report. Which of the following is the correct next action for the resident to take?",

"options": {

"A": "Disclose the error to the patient and put it in the operative report",

"B": "Tell the attending that he cannot fail to disclose this mistake",

"C": "Report the physician to the ethics committee",

"D": "Refuse to dictate the operative report"

},

"answer": "B"

},

"0001": {

"question": "A 67-year-old man with transitional cell carcinoma of the bladder comes to the physician because of a 2-day history of ringing sensation in his ear. He received this first course of neoadjuvant chemotherapy 1 week ago. Pure tone audiometry shows a sensorineural hearing loss of 45 dB. The expected beneficial effect of the drug that caused this patient's symptoms is most likely due to which of the following actions?",

"options": {

"A": "Inhibition of proteasome",

"B": "Hyperstabilization of microtubules",

"C": "Generation of free radicals",

"D": "Cross-linking of DNA"

},

"answer": "D"

Appendix C: Execution Logs Sample

C1 – Detailed Retrieval Log (Planner Contribution):

The following log entry demonstrates the Agentic Planner's ability to retrieve documents that were missed by the original query. In this instance, the document was found exclusively via the generated sub-query *"what is the difference between autophagy and aggrephagy"*, whereas the user's original phrasing failed to match the document.


```
{
 "question_id": "5e94902f2d3121100d000012",
 "rank": 1,
 "source_analysis": {
  "found_by_original": false,
  "found_by_planner": true,
  "planner_queries_count": 1
 },
 "all_matching_queries": [
  "what is the difference between autophagy and aggrephagy"
 ],
 "doc_content": "Aggrephagy is a type of autophagy that involves the degradation of aggregated proteins..."
}
```

The original query is below:

"5e94902f2d3121100d000012": {

"question": "Is aggrephagy a variant of autophagy?",

"options": {

"A": "yes",

"B": "no"

}

C2 – Evaluation Log: System Success (RAG Improved):

This sample illustrates a scenario where the Agentic RAG system successfully corrected a hallucination from the baseline LLM. The Judge Agent correctly identified the improvement, awarding the win to the RAG model based on factual adherence to the retrieved context.

{

"id": "17054994",

"status": "RAG_IMPROVED",

"question": "Does frozen section alter surgical management of multinodular thyroid disease?",

"correct_answer": "B",

"llm_ans": "A",

"rag_ans": "B",

"judge_winner": "RAG",

"judge_critique": "Model RAG's reasoning is more coherent and factual. It correctly identifies that there is no mention of frozen section in the provided context documents, which aligns with the correct answer B. Model LLM's reasoning forces irrelevant context into the answer."

}

C3 – Evaluation Log: Judge Bias (RAG Worsened):

This crucial example demonstrates the "Context Bias" phenomenon discussed in Chapter 4. Although the RAG system provided an incorrect answer (B) compared to the correct baseline (A), the Judge erroneously declared RAG the winner. The critique reveals that the Judge was misled by the "well-structured" nature of the hallucinated response.

{

"id": "5e94902f2d3121100d000012",

"status": "RAG_WORSENED",

"question": "Is aggrephagy a variant of autophagy?",

"correct_answer": "A",

"llm_ans": "A",

"rag_ans": "B",

"judge_winner": "RAG",

"judge_critique": "Model RAG's reasoning is well-structured and aligns with the correct medical answer... demonstrates a deep understanding. In contrast, Model LLM's reasoning is flawed... lacks coherence.",

"note": "Critical Failure: The Judge preferred the incorrect RAG answer over the correct LLM answer."

}